Spring 注解

1.配置组件

@Configuration和@Bean

@Configuration和@Bean注解配套使用，首先这个类是一个IOC容器，方法加了一个这个@Bean注解，一般这样的方法返回的都是一个对象，我们就可以直接从这个类中拿到这个实例，并且是单例的，我们可以通过AnnotationApplicationContext context = new AnnotationApplicationContext(加了@Configuration注解的类)去获取到这个bean对象

标注在类上，该类会被CGLIB动态代理生成子类，可以达到这样的效果：在某@Bean方法下调用另一个标注了@Bean的方法，得到的会是同一个Bean对象；

```
@Configuration
public class MyConfig {

    @Bean
    public Person person1(){
        return new Person("Tom",18);
    }
    // 添加一个方法
    @Bean
    public Person person(){
        return person1();
    }
}
```

```
// 比对两个bean
public class MyTest {
    @Test
    public void test() {
        ApplicationContext app = new
AnnotationConfigApplicationContext(MyConfig.class);
        Object bean = app.getBean("person");
        Object bean1 = app.getBean("person1");
        System.out.println(bean==bean1);

        String [] beanNames = app.getBeanNamesForType(Person.class);
        System.out.println(Arrays.toString(beanNames));

    }
}
```

@ComponentScan

该注解默认会扫描该类所在的报下所有的配置类

@Scope

@Scope("prototype")//多实例，IOC容器启动创建的时候，并不会创建对象放在容器在容器当中，当你需要的时候，需要从容器当中取该对象的时候，就会创建。
@Scope("singleton")//单实例 IOC容器启动的时候就会调用方法创建对象，以后每次获取都是从容器当中拿同一个对象（map当中）。
@Scope("request")//同一个请求创建一个实例
@Scope("session")//同一个session创建一个实例

@Lazy

延迟初始化

@Conditional(WinCondition.class)

条件主键，里面的类实现Condition，然后实现matches方法，返回的boolean值，如果为true,则执行该方法

@Import导入外部资源

带有@Configuration的配置类(4.2 版本之前只可以导入配置类，4.2版本之后 也可以导入 普通类)

1.新建一个TestA

```
public class TestA {

    public void fun(String str) {
        System.out.println(str);
    }

    public void printName() {
        System.out.println("类名 : " +Thread.currentThread().getStackTrace()
[1].getClassName());
    }
}
```

2.新建一个ImportConfig,在类上面加上@Configuration，加上@Configuration是为了能让Spring 扫描到这个类，并且直接通过@Import引入TestA类

```
@Import({TestA.class})
@Configuration
public class ImportConfig {
}
```

ImportSelector 的实现

1.新建TestC.class

```
public class TestC {
    public void fun(String str) {
        System.out.println(str);
    }

    public void printName() {
        System.out.println("类名 : " + Thread.currentThread().getStackTrace()
[1].getClassName());
    }
}
```

2.新建SelfImportSelector.class 实现ImportSelector 接口,注入TestC.class

```
public class SelfImportSelector implements ImportSelector {
    @Override
    public String[] selectImports(AnnotationMetadata importingClassMetadata) {
        return new String[]{"com.test.importdemo.TestC"};
    }
}
```

3.ImportConfig上面引入SelfImportSelector.class

```
@Import({TestA.class,TestB.class,SelfImportSelector.class})
@Configuration
public class ImportConfig {
}
```

ImportBeanDefinitionRegistrar 的实现

1.新建TestD.class

```
public class TestD {
    public void fun(String str) {
        System.out.println(str);
    }

    public void printName() {
        System.out.println("类名 : " + Thread.currentThread().getStackTrace()
[1].getClassName());
    }
}
```

2.新建SelfImportBeanDefinitionRegistrar.class,实现接口ImportBeanDefinitionRegistrar,注入TestD.class

```
public class SelfImportBeanDefinitionRegistrar implements
ImportBeanDefinitionRegistrar {
    @Override
    public void registerBeanDefinitions(AnnotationMetadata
importingClassMetadata, BeanDefinitionRegistry registry) {
        RootBeanDefinition root = new RootBeanDefinition(TestD.class);
        registry.registerBeanDefinition("testD", root);
    }
}
```

3.ImportConfig类上加上导入SelfImportBeanDefinitionRegistrar.class

```
@Import({TestA.class,TestB.class,SelfImportSelector.class,
        SelfImportBeanDefinitionRegistrar.class})
@Configuration
public class ImportConfig {
}
```

```
@Autowired
TestD testD;

@Test
public void TestD() {
    testD.printName();
}
```

BeanDefinitionRegistryPostProcessor

BeanDefinitionRegistryPostProcessor实现了BeanFactoryPostProcessor接口，是spring框架的BeanDefinitionRegistry的后处理器，用来注册额外的BeanDefinition

实现factoryBean接口

FactoryBean 就是 让我们自己 来创建一个对象，FactoryBean看起来是没有多大的好处

什么时候使用?

如果这里的创建步骤特别复杂，或需要计算后 才能得到一个对象。就可以使用

既然是根据一些数据创建对象，为什么非要写到FactoryBean中，而不是直接写一个方法

使用FactoryBean与 自己创建对象对比

但是调用端发生了不同之处: 可以直接注入使用!

对比之间的区别, FactoryBean总结如下:

1. 对于调用方只需要知道需要对象即可, 不需要管生产方式, 符合迪米特法则: 知道的越少越好, 减少耦合
2. 通过FactoryBean创建的对象与其他正常对象 使用起来毫无差异, 不需要区别对待
3. 将创建对象的复杂操作, 全部封装到了FactoryBean接口的实现类中, 这个类更具语义化, 让大家一看就知道它要做什么事情, 并与Spring的编码风格进行统一

总结:

单独的使用对象实现FactoryBean接口往往收益不大, 常用的都是同时继承其他类,实现其他接口来协作使用

1. 创建对象时需要依赖于 别的接口 监听数据,推送数据过来的接口 做一些数据处理时, 可以使用FactoryBean.
2. 需要搭配其他接口协作, 依赖于Spring的某个生命周期内, 某个时间节点 来生成对象

自定义注解

- 先定义类上面的注解

```
// 表示作用在类上
@Target({ElementType.TYPE})
// 运行时的注解
@Retention(RetentionPolicy.RUNTIME)
// @Documented 将此注解包含在javadoc 中
@Documented
// 代表是spring的注解，会被spring扫描到
@Component
public @interface GupaoAnnotionClass {

    boolean value() default false;
}
```

```
// 表示作用在方法上
@Target({ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Component
public @interface GupaoAnnotionMethod {

    String value() default "wudasheng";
}
```

```
@GupaoAnnotionClass
public class Wudasheng {

    @GupaoAnnotionMethod
    public void study(){
        System.out.println("study spring");
    }
}
```

```
@Configuration
@ComponentScans({
        @ComponentScan("com.gupaoedu.project.gupaoentity"),
        @ComponentScan("com.gupaoedu.demo.annotaions.configures.gupaoannotion")
})
public class MyConfig {

}
```

```
public class MyTest {
    @Test
    public void test() {
        ApplicationContext app = new
AnnotationConfigApplicationContext(MyConfig.class);

        Wudasheng wudasheng = (Wudasheng)app.getBean("wudasheng");
        wudasheng.study();
    }
}
```

```
@Component
public class GupaoBeanPostProcessor implements BeanPostProcessor {

    @Override
    public Object postProcessAfterInitialization(Object bean, String beanName)
throws BeansException {
        Object instance = null;
        if(bean.getClass().isAnnotationPresent(GupaoAnnotionClass.class)){
            GupaoAnnotionClass annotation =
bean.getClass().getAnnotation(GupaoAnnotionClass.class);
            if(annotation.value()){
                return bean;
            }
            Method[] declaredMethods = bean.getClass().getDeclaredMethods();
            for(Method method : declaredMethods){
                if(method.isAnnotationPresent(GupaoAnnotionMethod.class)) {
                    GupaoAnnotionMethod annotionMethod =
method.getAnnotation(GupaoAnnotionMethod.class);
                    if("Wudasheng".equals(annotionMethod.value())){
                        instance = getInstance(bean.getClass());
                    }
                }
            }
        }
        return instance;
    }

    public Object getInstance(Class<?> clazz) throws BeansException {
        //相当于Proxy，代理的工具类
        // 生成的代码是代理类的子类
        Enhancer enhancer = new Enhancer();
        enhancer.setSuperclass(clazz);
        enhancer.setCallback(new MethodInterceptor() {
            @Override
            public Object intercept(Object o, Method method, Object[] objects,
MethodProxy methodProxy) throws Throwable {
```

```
                System.out.println("准备开始学习");
                Object obj = methodProxy.invokeSuper(o,objects);
                System.out.println("一天就学会了");
                return obj;
            }
        });
        return enhancer.create();
    }
}
```